

Network Programming 2026 - Assignment #3

Course: Network Programming

Assignment: Assignment #3 (Chapters 7, 8)

Student ID: 20221528

Question 1. Async Chat Relay with Channels

1. Screenshot

```
PS C:\Users\akftj\Downloads\Network Programming 2026_ Assignment 3_templates\20221528> python solution1_server.py
Server listening on 127.0.0.1:9000
JOIN 127.0.0.1:51403 -> study
MESSAGE 127.0.0.1:51403 [study]: hello!
MESSAGE 127.0.0.1:51403 [study]: i'm first
JOIN 127.0.0.1:51418 -> study
MESSAGE 127.0.0.1:51418 [study]: wow!
MESSAGE 127.0.0.1:51418 [study]: i'm second
JOIN 127.0.0.1:51425 -> gaming
MESSAGE 127.0.0.1:51425 [gaming]: hello!
MESSAGE 127.0.0.1:51425 [gaming]: every here?
QUIT 127.0.0.1:51403 from study
LEAVE 127.0.0.1:51403 <- study
^

PS C:\Users\akftj\Downloads\Network Programming 2026_ Assignment 3_templates\20221528> python solution1_client.py
Channel: study
> Joined channel: study
hello!
> i'm first
> [127.0.0.1:51418] wow!
[127.0.0.1:51418] i'm second
quit
PS C:\Users\akftj\Downloads\Network Programming 2026_ Assignment 3_templates\20221528> ^

PS C:\Users\akftj\Downloads\Network Programming 2026_ Assignment 3_templates\20221528> python solution1_client.py
Channel: study
> Joined channel: study
wow!
> i'm second
> [127.0.0.1:51403] left the channel
^

PS C:\Users\akftj\Downloads\Network Programming 2026_ Assignment 3_templates\20221528> python solution1_client.py
Channel: gaming
> Joined channel: gaming
hello!
> every here?
> ^
```

Figure 1 shows one server and three clients running at the same time. Two clients joined the study channel, and one client joined the gaming channel. The study clients exchanged messages with each other, while the gaming client did not receive the study channel messages. When one study client typed quit, the server removed that client from the channel and notified the remaining study client.

2. My Code Logic

For this question, I used asyncio streams to make a TCP chat server. The server listens on 127.0.0.1:9000. When a client connects, the first line sent by the client is treated as the channel name. I stored channel information in a dictionary named channels. Each key is a channel name, and each value is a set of connected StreamWriter objects in that channel.

After a client joins a channel, the server reads later lines as chat messages. The broadcast function sends the message only to other clients in the same channel. The sender is skipped, so a client does not receive its own message back. This also keeps messages from one channel from being sent to another channel.

The client connects to the server, sends the channel name first, and then starts receiving messages from the server. To handle user input and server messages at the same time, I used one asyncio task for receiving network messages and used asyncio.to_thread for sys.stdin.readline. This prevents terminal input from blocking incoming server messages.

3. Exception Handling and Concurrency Strategy

The server handles each connected client with an asyncio coroutine, so no threads or multiprocessing are used on

the server side. If a client sends quit, disconnects unexpectedly, or sends invalid data, the finally block calls

remove_client. This removes the client writer from the channel set, deletes the empty channel if necessary, and

closes the connection.

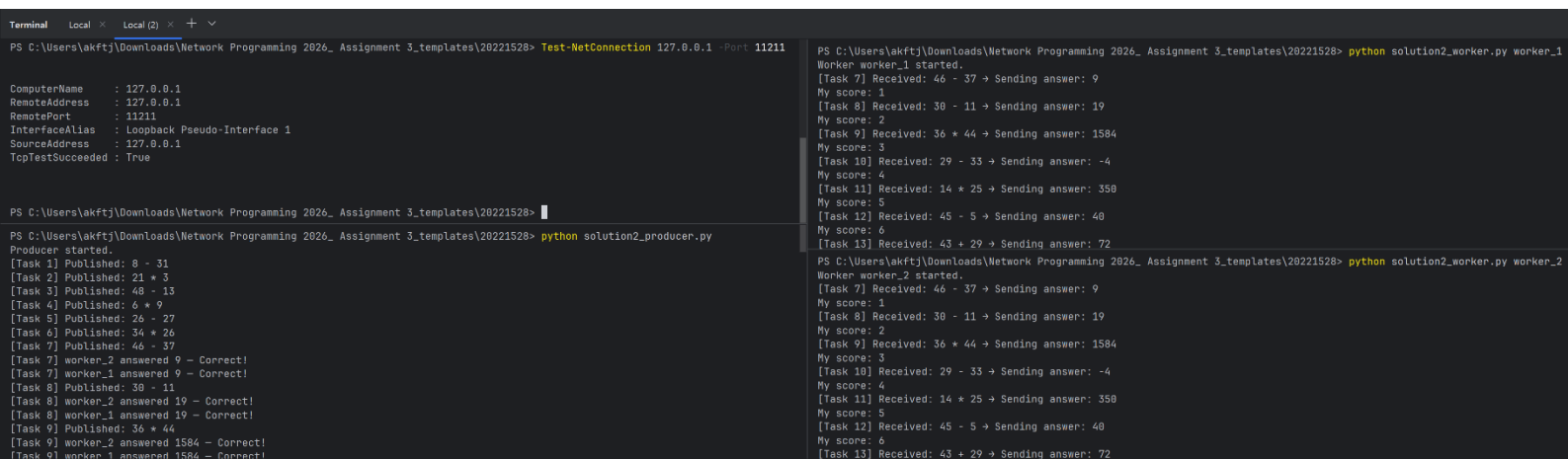
The server catches UnicodeDecodeError for invalid UTF-8 data and socket-related errors such as ConnectionError,

OSError, and RuntimeError. These errors are logged, but they do not stop the entire server. On the client side,

KeyboardInterrupt and network errors are also handled so the socket can be closed cleanly.

Question 2. Pub/Sub Task Queue with Memcached Scoreboard

1. Screenshot



```
Terminal Local x Local (2) x + v
PS C:\Users\akftj\Downloads\Network Programming 2026_ Assignment 3_templates\20221528> Test-NetConnection 127.0.0.1 -Port 11211

ComputerName : 127.0.0.1
RemoteAddress : 127.0.0.1
RemotePort : 11211
InterfaceAlias : Loopback Pseudo-Interface 1
SourceAddress : 127.0.0.1
TcpTestSucceeded : True

PS C:\Users\akftj\Downloads\Network Programming 2026_ Assignment 3_templates\20221528>
PS C:\Users\akftj\Downloads\Network Programming 2026_ Assignment 3_templates\20221528> python solution2_producer.py
Producer started.
[Task 1] Published: 8 - 31
[Task 2] Published: 21 * 3
[Task 3] Published: 48 - 13
[Task 4] Published: 6 * 9
[Task 5] Published: 26 - 27
[Task 6] Published: 34 * 26
[Task 7] Published: 46 - 37
[Task 7] worker_2 answered 9 - Correct!
[Task 7] worker_1 answered 9 - Correct!
[Task 8] Published: 30 - 11
[Task 8] worker_2 answered 19 - Correct!
[Task 8] worker_1 answered 19 - Correct!
[Task 9] Published: 36 * 44
[Task 9] worker_2 answered 1584 - Correct!
[Task 9] worker_1 answered 1584 - Correct!

PS C:\Users\akftj\Downloads\Network Programming 2026_ Assignment 3_templates\20221528> python solution2_worker.py worker_1
Worker worker_1 started.
[Task 7] Received: 46 - 37 -> Sending answer: 9
My score: 1
[Task 8] Received: 30 - 11 -> Sending answer: 19
My score: 2
[Task 9] Received: 36 * 44 -> Sending answer: 1584
My score: 3
[Task 10] Received: 29 - 33 -> Sending answer: -4
My score: 4
[Task 11] Received: 14 * 25 -> Sending answer: 350
My score: 5
[Task 12] Received: 45 - 5 -> Sending answer: 40
My score: 6
[Task 13] Received: 43 + 29 -> Sending answer: 72

PS C:\Users\akftj\Downloads\Network Programming 2026_ Assignment 3_templates\20221528> python solution2_worker.py worker_2
Worker worker_2 started.
[Task 7] Received: 46 - 37 -> Sending answer: 9
My score: 1
[Task 8] Received: 30 - 11 -> Sending answer: 19
My score: 2
[Task 9] Received: 36 * 44 -> Sending answer: 1584
My score: 3
[Task 10] Received: 29 - 33 -> Sending answer: -4
My score: 4
[Task 11] Received: 14 * 25 -> Sending answer: 350
My score: 5
[Task 12] Received: 45 - 5 -> Sending answer: 40
My score: 6
[Task 13] Received: 43 + 29 -> Sending answer: 72
```

Figure 2 shows Memcached running on 127.0.0.1:11211, one producer, and two workers.

The producer published

math tasks through ZeroMQ PUB. Both workers received the tasks through SUB, calculated the answers, and sent

the answers back through PUSH. The producer received the answers through PULL, checked correctness, and

updated each worker score in Memcached. The worker scores increased after each correct answer.

2. My Code Logic

The producer creates random math tasks using the operators +, -, and * with operands from 1 to 50. Each task is

stored as a JSON object with task_id, operand1, operator, and operand2. The producer sends this JSON task

through a ZeroMQ PUB socket bound to tcp://127.0.0.1:5555.

The workers use ZeroMQ SUB sockets to receive every published task. Each worker computes the correct answer

locally and creates an answer JSON object containing task_id, worker_id, and answer. The answer is sent back to

the producer through a ZeroMQ PUSH socket connected to tcp://127.0.0.1:5556.

The producer uses a PULL socket to collect worker answers. It keeps a dictionary named `expected_answers` so it

can check each submitted answer. If the answer is correct, the producer reads the current worker score from

Memcached, adds one point, and stores the updated score under the key `score:<worker_id>`. The `scored_answers`

set prevents the same worker from being scored twice for the same task.

3. Memcached Installation and Start Command

Memcached must be running locally on the default port 11211 before running the producer and workers. I

checked the connection with `Test-NetConnection` and confirmed `TcpTestSucceeded` was `True`.

Commands used

```
pip install pyzmq pymemcache
```

```
docker run --name cn-a3-memcached --rm -d -p 11211:11211 memcached:latest
```

```
Test-NetConnection 127.0.0.1 -Port 11211
```

4. Exception Handling and Concurrency Strategy

The producer must publish tasks and receive answers without blocking on only one operation. To do this, I used

zmq.Poller with a short timeout. In each loop, the producer first checks whether it is time to publish a new task,

and then polls the PULL socket for available answers. When answers are waiting, the producer drains the socket

using a non-blocking receive.

The producer checks malformed JSON, missing answer fields, unknown task IDs, duplicate answers, and

Memcached read/write errors. The worker also handles malformed tasks and missing fields without stopping. Both

producer and worker catch KeyboardInterrupt so that Ctrl+C shuts the programs down cleanly and closes the

ZeroMQ sockets and Memcached client.

Short Summary

In Question 1, I implemented a channel-based TCP chat relay using asyncio. The server manages multiple clients

concurrently in one event loop, and messages are relayed only inside the correct channel. In Question 2, I

implemented a task distribution system using ZeroMQ PUB/SUB for tasks, PUSH/PULL for answers, and

Memcached for per-worker scores. The code was added after the original template code without changing the

original template section.